北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

代理式编程 AGENTIC CODING

——从大模型，到会自己写代码的智能体

授课人：闫波

时间：2026/9/21

“自然语言，正在成为新的编程语言。”

过去：查文档 → 写代码 → 调试，耗时长；

现在：一句话描述需求 → Agent 自主规划、编代码、跑测试、修 Bug。

北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

目录

CONTENTS

1 人工智能的发展

2 代理式编程的发展与分类

3 常见的代理式编程工具

4 代理式编程的开发流程

北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

1 人工智能的发展

人工智能的发展历程



符号主义

1950s–70s
逻辑推理、专家规则；早期乐观后遇上第一次 AI 寒冬。

机器学习

1980s–2000s
统计学习、SVM、专家系统；从人定规则转向在数据中找规律。

深度学习

2010s–至今
数据、算力、算法三个要素得到长足发展，人工智能热潮爆发。

前两次发展都遇到障碍，但是这一次热度没有减退，
这是理解今天 AI 爆发的起点。

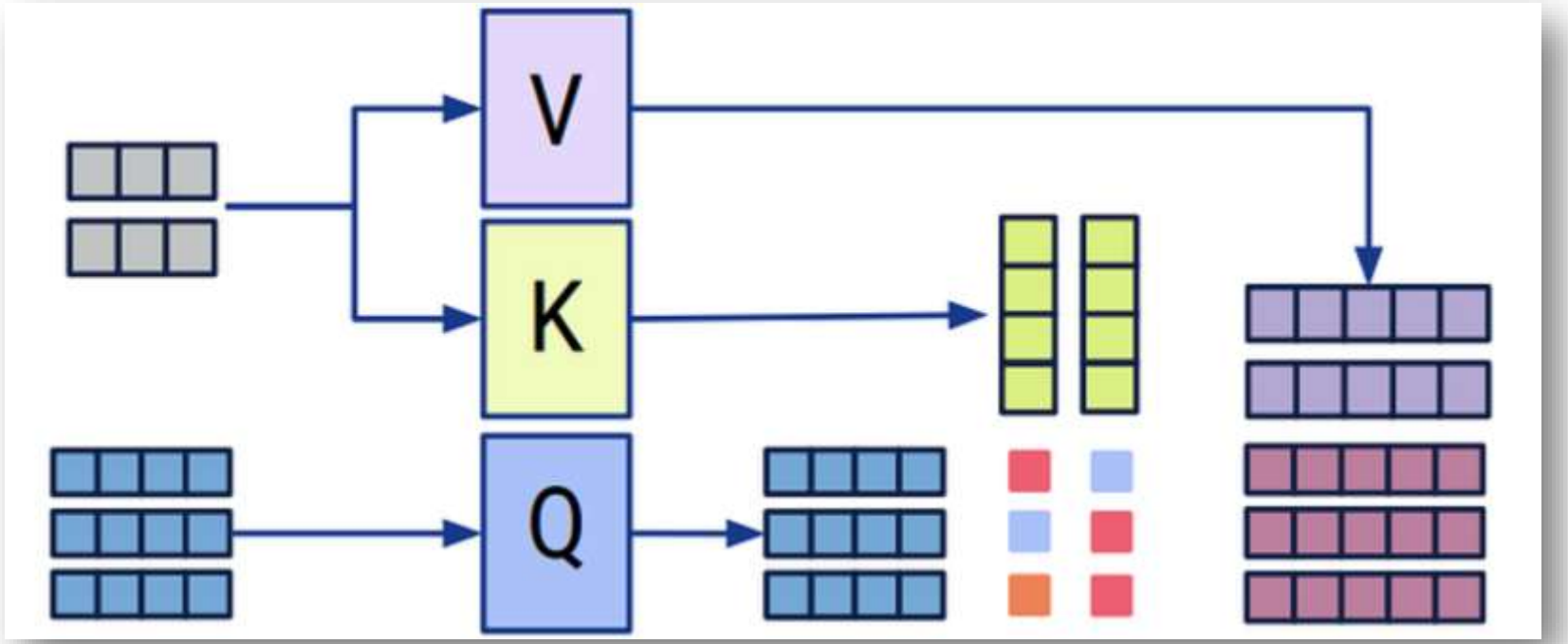
2012年深度学习引擎

- **神经网络**：AlexNet 在 ImageNet 图像识别大赛上大幅领先，神经网络回归到大众视野。
- **核心思想**：端到端、自动从数据中学习特征，而非人工设计特征。
- **全面突破**：CNN / RNN 在图像、语音、机器翻译上接连刷新纪录。



关键转变：从人来设计特征到模型自己学特征，这也是后来大模型能力涌现的基础。

Transformer 与注意力机制



2017 年出现

论文《Attention Is All You Need》提出 Transformer 架构。

可大规模并行

用自注意力取代循环结构，训练能极致并行，把模型做大成为可能。

现代大模型的底座

GPT、BERT、Claude、Gemini…绝大多数都建立在它之上。

ChatGPT 的出现与发展

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

2018–20

GPT-1 / 2 / 3
参数与数据逐级放大，语言能力持续增强，但仍是研究者的工具。

2022.11

ChatGPT 发布 — 转折点
对话式交互 + 强指令遵循，史上最快破亿用户级应用之一，AI 进入全民视野。

关键

预训练 + 指令微调 + RLHF
让模型既拥有大量知识又遵循指令，能按人的意图办事。

ChatGPT 的真正价值，并不在于模型能力的进一步增强，而在于它首次让普通人能够通过自然语言直接驱动 AI。交互门槛的显著降低，才是其爆发式发展的关键诱因。

ChatGPT 之后的发展



模型更大

各厂相继推出更强模型（GPT、Claude、Gemini...），上下文窗口从几千token涨到百万级。



模态更多

从纯文本，扩展到图像、音频、视频、代码全面打通。进阶到多模态模型时代，不再受局限。

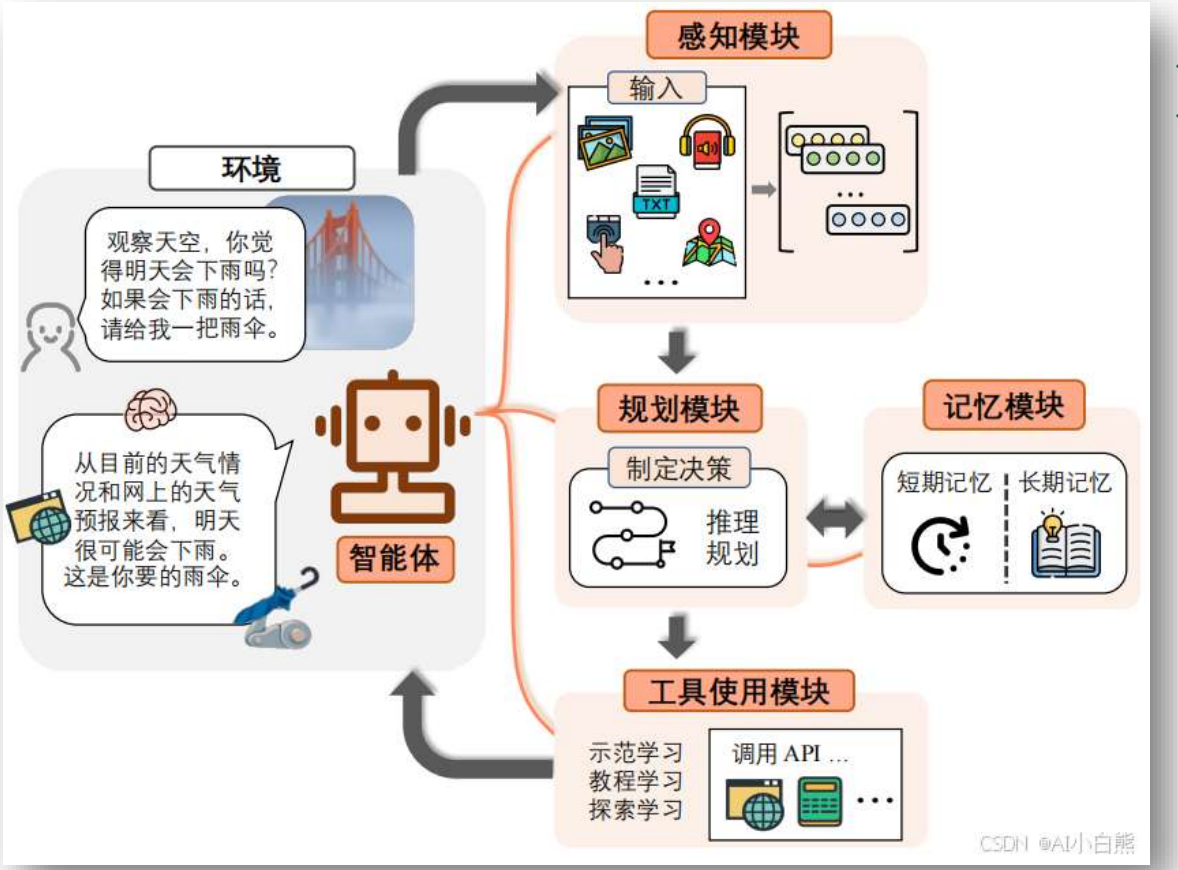


能力更强

模型能力更加强大，推理、规划、工具使用等能力不断随规模涌现，这也是代理式编程的前提。

工具使用和规划这两项能力，是后面 Agent 能够自我工作的关键前提。

大模型与执行能力

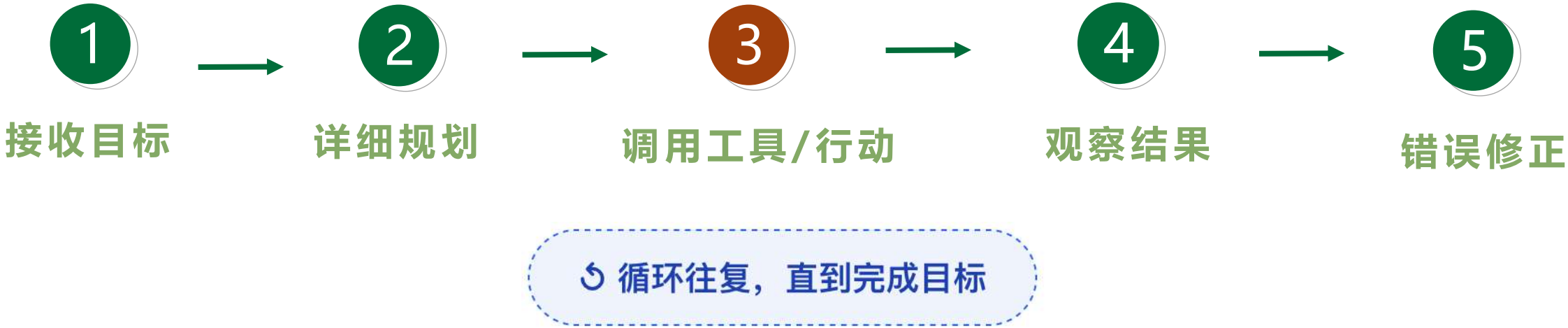


当模型能自己决定调用什么工具、并根据结果继续行动时，就出现了智能体（Agent）。

仅具备对话能力并不足够，关键在于能够真正完成任务。能力落地主要包括以下几条路径：

- **提示工程（Prompt）**：通过更优质的指令引导模型输出。
- **检索增强（RAG）**：使模型能够调用外部知识和最新资料。
- **工具调用（Function Calling）**：让模型具备调用API、查询数据库和执行代码的能力。
- **插件与生态**：将模型能力接入真实世界的业务系统。

Agent 的出现



与传统对话模型的本质区别：**多步骤、自主、带反馈循环**。

类比：导航 App 规划路线、自动驾驶直接开到目的地。

为什么编程是 AI 的最佳应用场景

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

结构化可验证

代码可以**编译**、**测试**，**反馈**，Agent 可以验证正确性。

多步骤可循环

开发天然是编码、运行、修改的循环任务，契合 Agent 闭环。

工具链成熟

终端、Git、编译器、测试框架都可被 Agent 直接调用。

语料多收益大

训练数据丰富，且直接转化为生产力提升。

可验证 + 可循环 是 Agent 能在编程任务中工作的根本原因。

当 Agent 结合 代码

让 AI 智能体自主地规划、编写、运行、调试代码来完成开发任务——

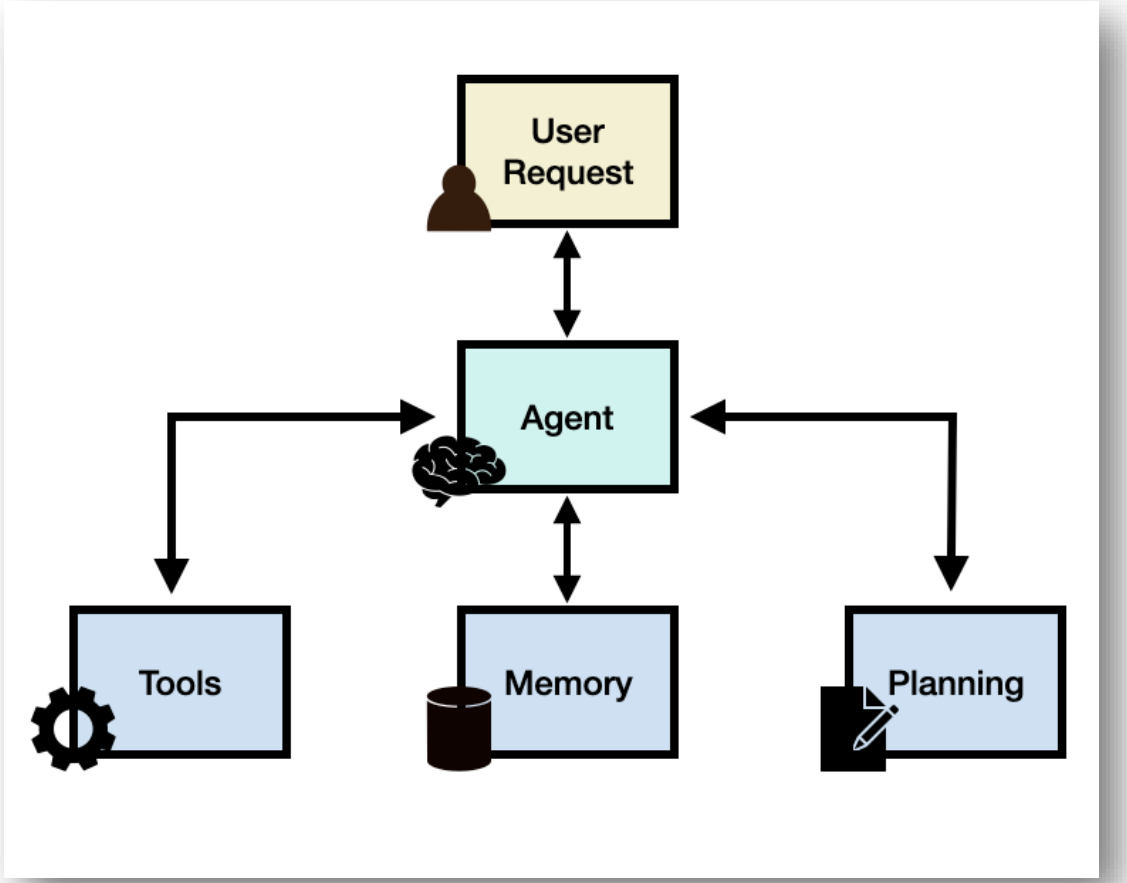
这就是**代理式编程（Agentic Coding）**。

2 代理式编程的发展与分类

定义：从自动补全到自主开发

代理式编程：AI 智能体在给定目标下，**自主完成**理解需求、规划任务、读写多文件代码、执行命令、运行与修复测试等开发动作的**编程范式**。

判断标志：交办一个任务，其可以**自行运行一串步骤**，而不只是给一行建议。



核心机制：自主理解任务，独立完成相应工作



上下文管理

理解整个项目的结构与逻辑，明确需要修改的位置以及具体的修改方式。

长程规划与记忆

能够将复杂任务拆解为多个步骤，并持续记录当前进展与完成状态

工具使用

调用终端 / 文件 / Git / 浏览器等真实工具。

发展历程：从补全、对话到代理



补全时代

~2021 前后
代码自动补全为代表，行级 / 块级建议，补全一行代码。

对话时代

2022–2023
对话式问答、解释、生成片段，进入 IDE 侧边栏，回答一段内容。

代理时代

2024–至今
自主多步执行、跨文件改动、跑命令与测试，做一串事，并迅速分化。

能力边界不断拓展：从补全一行代码，到回答一段内容，再到完成一整类任务。其最终指向，是能力形态的分化，因此也形成了后续的分类。

代理式编程的分类依据

代理式编程出现后，很快从单一形态走向多元分化。不同产品之间的差异，不再只是模型能力或代码补全效果的差异，而更多体现在用户从哪里进入，以及如何与 Agent 协作。因此，理解代理式编程工具，不能只看背后的模型和厂商，更要看它所处的使用场景、交互方式和协作边界。

课程采用的分类依据：按**使用入口 / 交互形态**分类，而非按厂商或模型。

原因是对于开发者而言，**工具使用的场景与方式**，比其背后采用哪一家模型，更影响开发体验。

提示：同一厂商可能覆盖多个类别（例如 Claude Code 同时出现在多种分类中）。因此，分类依据的是产品形态，而非厂商归属。

按使用入口 / 交互形态分四类

命令行

Terminal Agent。
入口：终端、TUI、shell

IDE / 编辑器

原生 Agent。入口：类 VS Code 的 IDE、独立编辑器

传统 AI 助手

（不完全 agentic）
入口：IDE 的插件。

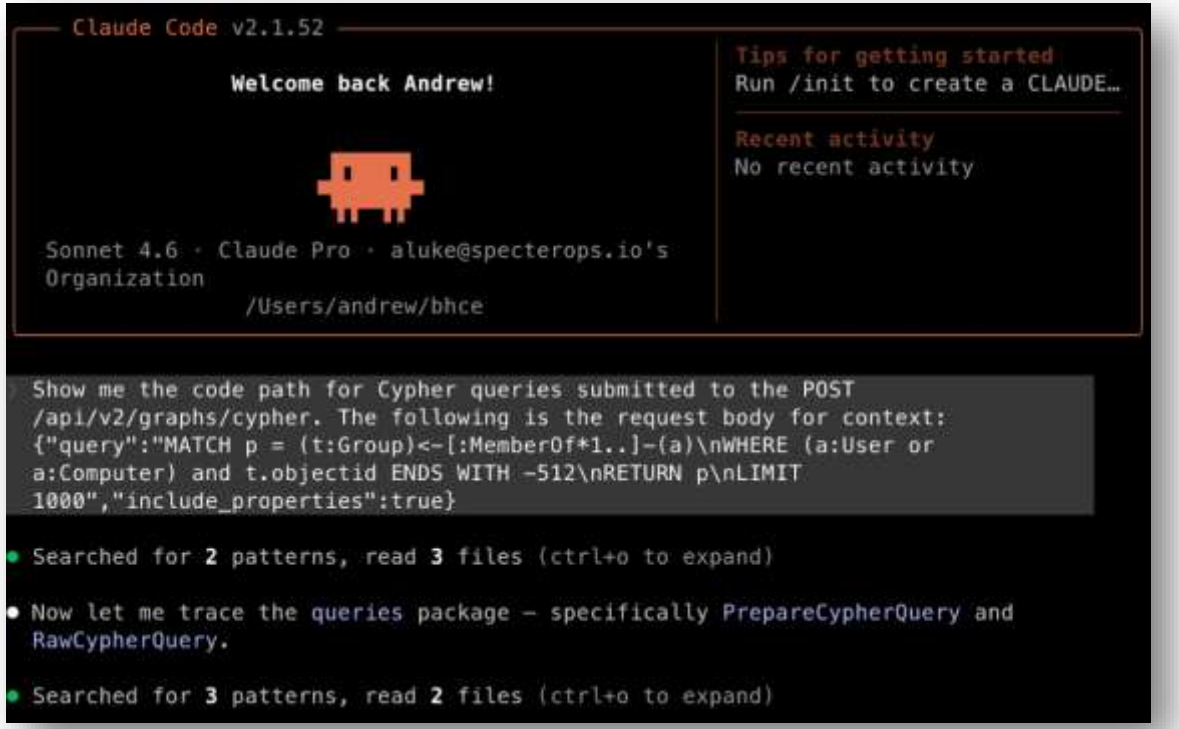
桌面工作台

Agent 工作台。入口：独立应用，管理多个 agent 任务

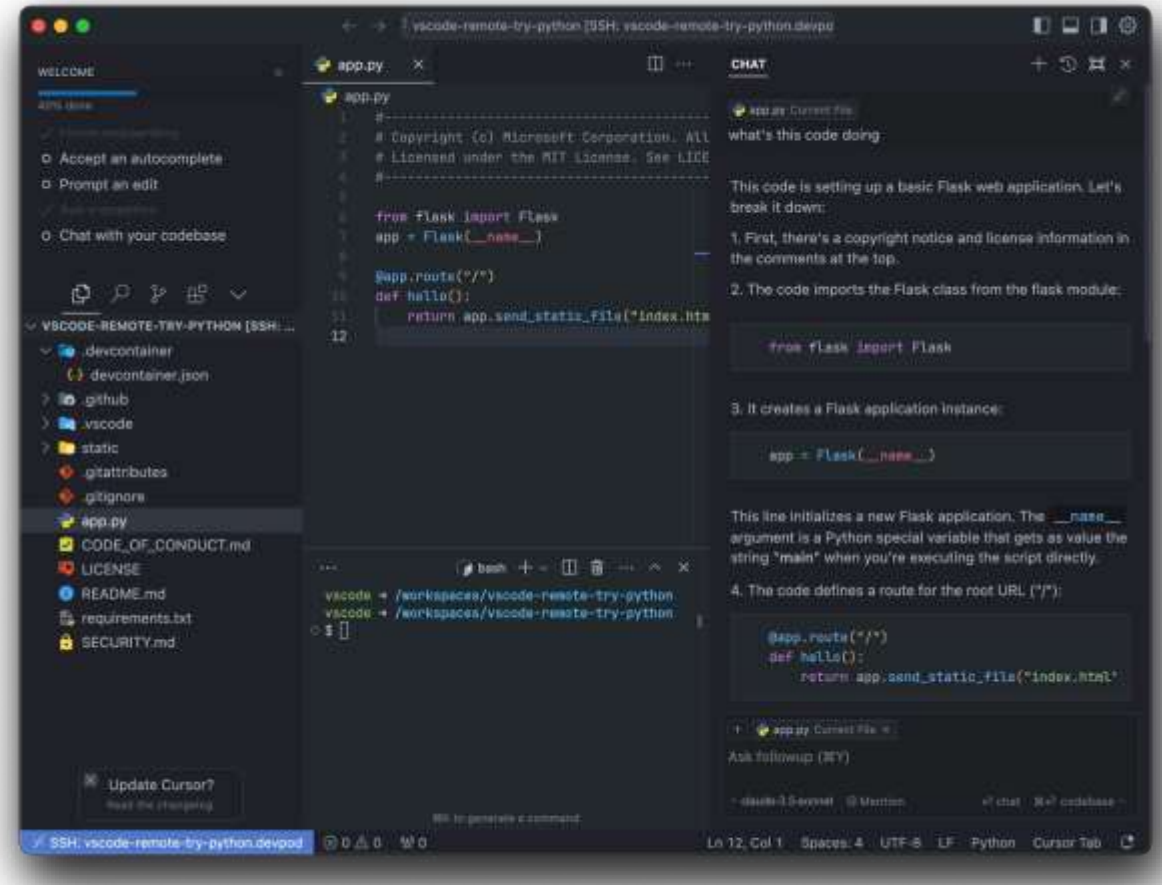
命令行中的 Agent

- **特点：**轻量、无 GUI 依赖；与 Git、命令行工具、CI 环境天然契合；可脚本化、可在远程/容器内运行。
- **擅长任务：**批量自动化改动、脚本与运维、远程服务器开发、嵌入工作流与流水线、重度命令行用户的高效操作。

命令行最接近系统、最易自动化。不绑定某个编辑器，优点在于通用与可组合。



IDE/编辑器中的 Agent

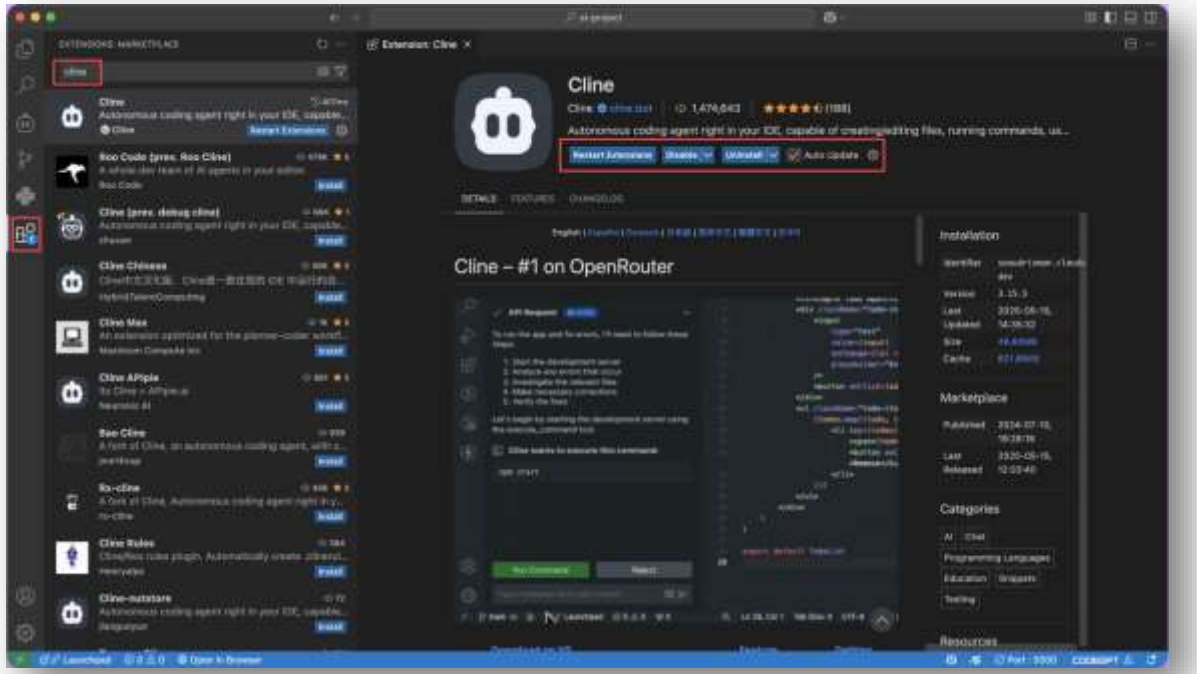


- **特点：** Agent 是产品的原生功能，与编辑、调试、文件树、Git 深度融合；可视化 diff、即时预览、补全与对话一体；上手平滑。
- **擅长任务：** 日常功能开发、需频繁人工审阅微调的工作、前端/全栈可视化开发、习惯图形化 IDE 的开发者。

第一类是把 Agent 做进命令行，第二类是把编辑器重做成 Agent 优先，是当下的主流形态。

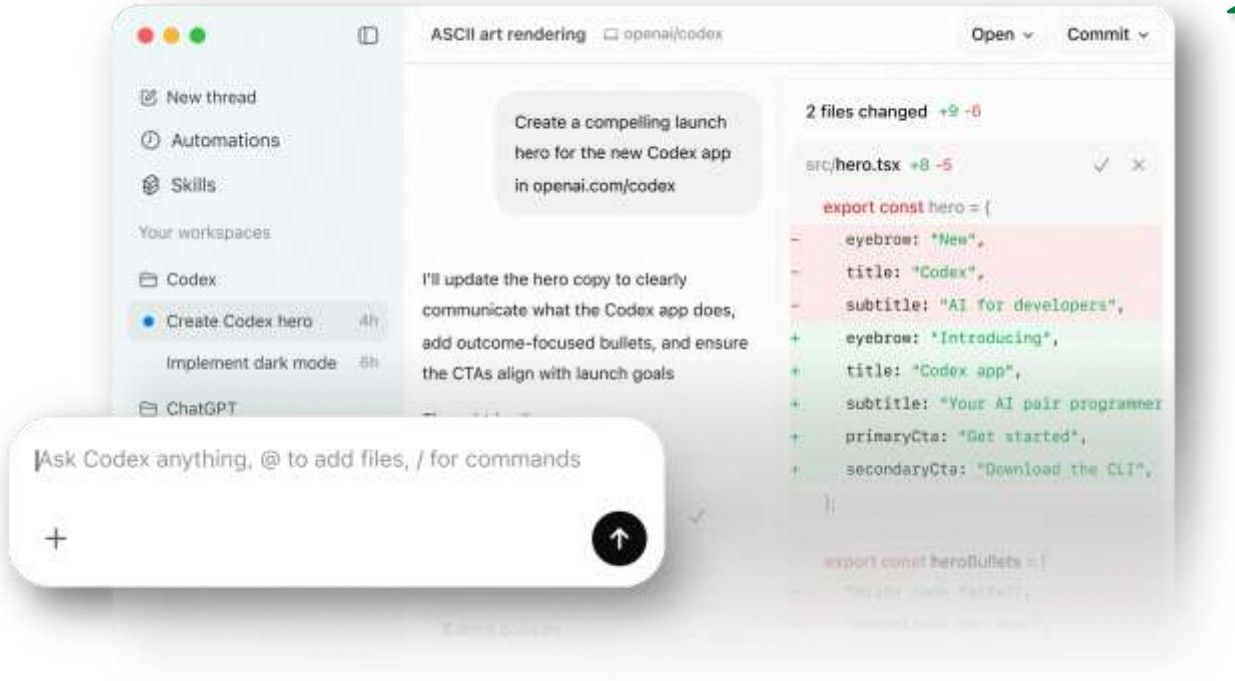
以插件形态存在的智能助手

- **特点：**起步于自动补全/对话辅助，以辅助人为主、人始终主导；近年加入Agent 模式，但自主程度常弱于类型一和类型二。
- **优势：**无缝嵌入既有工具链、生态成熟、企业接受度高。
- **擅长任务：**不改变现有环境下提效，如补全、解释、生成测试、渐进式引入 AI。



属于不完全 agentic，生成建议、并局部修改代码，但默认不会自己跑一系列任务。常见于团队平滑过渡到 AI 编程的起点，且正快速进化出 Agent 能力，边界在模糊。

桌面端 Agent 控制台



- **特点：**独立桌面应用，像任务看板一样并行管理多个 Agent 编程任务，处理多项目、多分支、异步运行、可视化进度。
- **擅长任务：**同时推进多任务、长时间异步作业、个人/团队的Agent 流水线管理，可进行任务编排（Orchestration）。

四类代理式编程工具对比

维度	1、命令行	2、编辑器原生	3、传统插件	4、桌面工作台
入口形态	终端/TUI	原生IDE/编辑器	现有IDE插件	独立桌面应用
自治程度	**高**	中-高	低-中	**高（多Agent）**
人机协作紧密度	中	**高**	**高**	中（偏异步）
自动化能力	**强**	中	弱-中	**强**
擅长场景	自动化/远程/运维	日常开发/审阅	渐进提效/不换环境	多任务并行编排
上手门槛	较高	低	**最低**	中

每类代理式编程工具都有各自的特色和擅长的任务，多尝试、多验证，选出最适合自己学习工作的一款最重要。



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

3 常见的代理式编程工具

常见的代理式编程工具

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY



命令行

- Claude Code CLI
- Codex CLI
- Gemini CLI
- OpenCode



编辑器原生

- Cursor
- Trae
- Antigravity



IDE 插件

- Claude Code（插件）
- Cline
- GitHub Copilot



桌面工作台

- Claude Code Desktop
- Codex Desktop

Anthropic：从 AI 安全出发的前沿模型公司

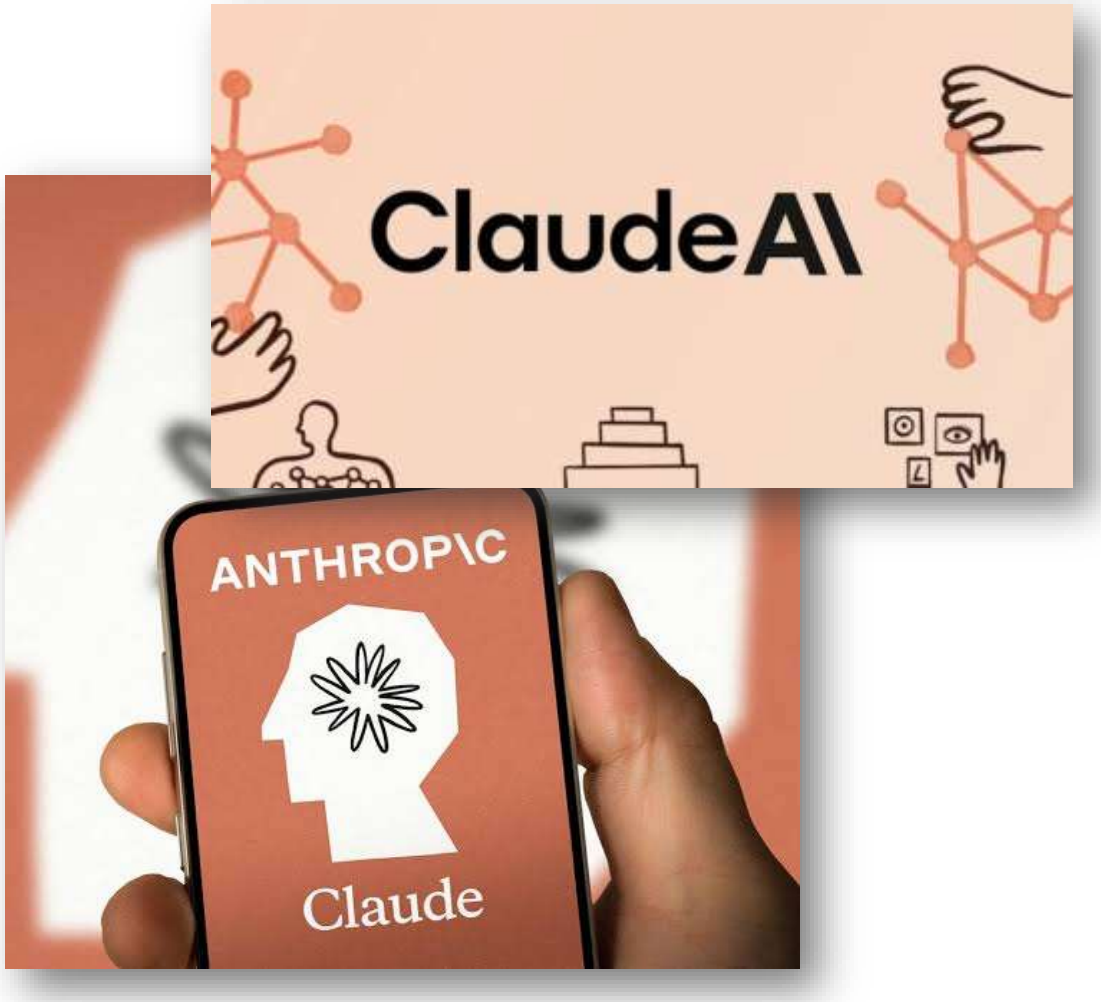
北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

Anthropic 将自身定位为一家 **AI 安全与研究公司**，核心目标是在提升模型能力的同时，让人工智能更加可靠、可解释和可控。围绕这一理念，公司持续推进 **Claude 系列模型**，并将安全研究直接融入模型训练和产品部署，逐渐形成了区别于其他大模型公司的技术路线与品牌定位。





Anthropic：Claude 从聊天助手走向智能体



近年来，Anthropic 的重点从通用对话模型进一步延伸到**编程、知识工作和企业级智能体**。Claude Code 等产品让 Claude 可以直接参与软件开发和复杂工作流，2026 年又持续推出新一代 Claude 模型，并强化**企业安全、云平台合作与行业落地**，逐渐从“模型提供商”走向企业 AI 基础设施平台。Anthropic 最新发布的 **Claude Fable 5.1 与 Claude Mythos 5.1** 在深度推理、Agent 任务及科研辅助上实现了重大突破。

OpenAI：从大模型研究走向大众化 AI

OpenAI 的发展推动了**生成式 AI 从研究领域快速进入大众市场**。以 GPT 系列为技术基础，ChatGPT 将自然语言交互变成普通用户使用 AI 的主要入口，随后 OpenAI 不断扩展模型在推理、编程、多模态和科学研究等领域的能力，使 AI 从“内容生成工具”逐步升级为能够**处理复杂任务**的智能助手。近日报道称，OpenAI 的最新估值已攀升至 8520 亿美元。





OpenAI：从 ChatGPT 走向 AI 工作平台





OpenAI 当前的发展重点已经不只是提升单一模型能力，而是**围绕 ChatGPT、Codex、API 和企业产品**构建完整的 AI 应用生态。新一代 GPT 模型持续强化知识工作、编程、科学研究与智能体能力，同时更加**关注推理效率和使用成本**，目标是让 AI 更深地参与个人与企业的实际工作流程。近日，OpenAI 刚刚发布了其迄今为止最强大的下一代 AI 模型 **GPT-6 Astra**，标志着 OpenAI 的技术正式迈向了其定义的 AGI 时代。

Google：从长期 AI 研究走向 Gemini 时代

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

Google 在搜索、机器学习和 DeepMind 等领域**拥有长期积累**，并在 2023 年正式推出 **Gemini**，进一步整合 Google 与 DeepMind 的 AI 研发能力。Gemini 从一开始就强调原生多模态，可以同时理解文本、图像、音频、视频和代码，也成为 Google 将**下一代 AI 技术**统一到一个核心模型体系中的重要转折点





Google：用 AI 重构整个 Google 生态



Google 的核心优势在于能够把 Gemini 快速嵌入 Search、Workspace、Android、YouTube、Cloud 等庞大的**产品生态**。到 2026 年，Gemini 进一步向智能体方向发展，新一代模型更加重视编程、复杂推理和自动执行任务，使 Google 的 AI 战略从“提供模型”转向“让 AI 成为整个产品生态的智能层”。谷歌近日正式发布了 **Gemini 3.8 Flash 与 Gemini 3.8 Flash Cyber** 两款新模型，大幅提升了长周期编程与智能体的自主执行能力。

命令行 Agent



Claude Code CLI



Codex CLI



Gemini CLI



OpenCode

共同点：均在**命令行**中运行，均可读取代码库、修改文件、执行命令、运行测试。

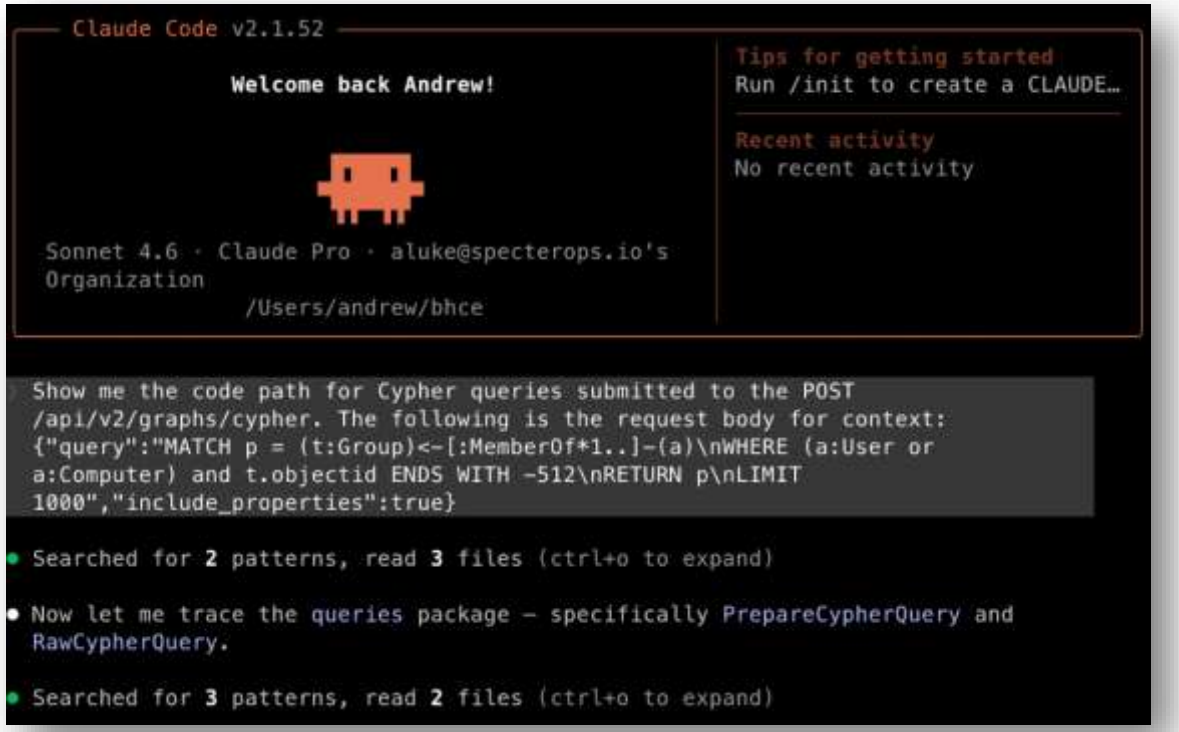
关注差异：背后模型、开放程度（开源/闭源）、生态与扩展、免费友好度。

Claude Code CLI

闭源 Claude 模型

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

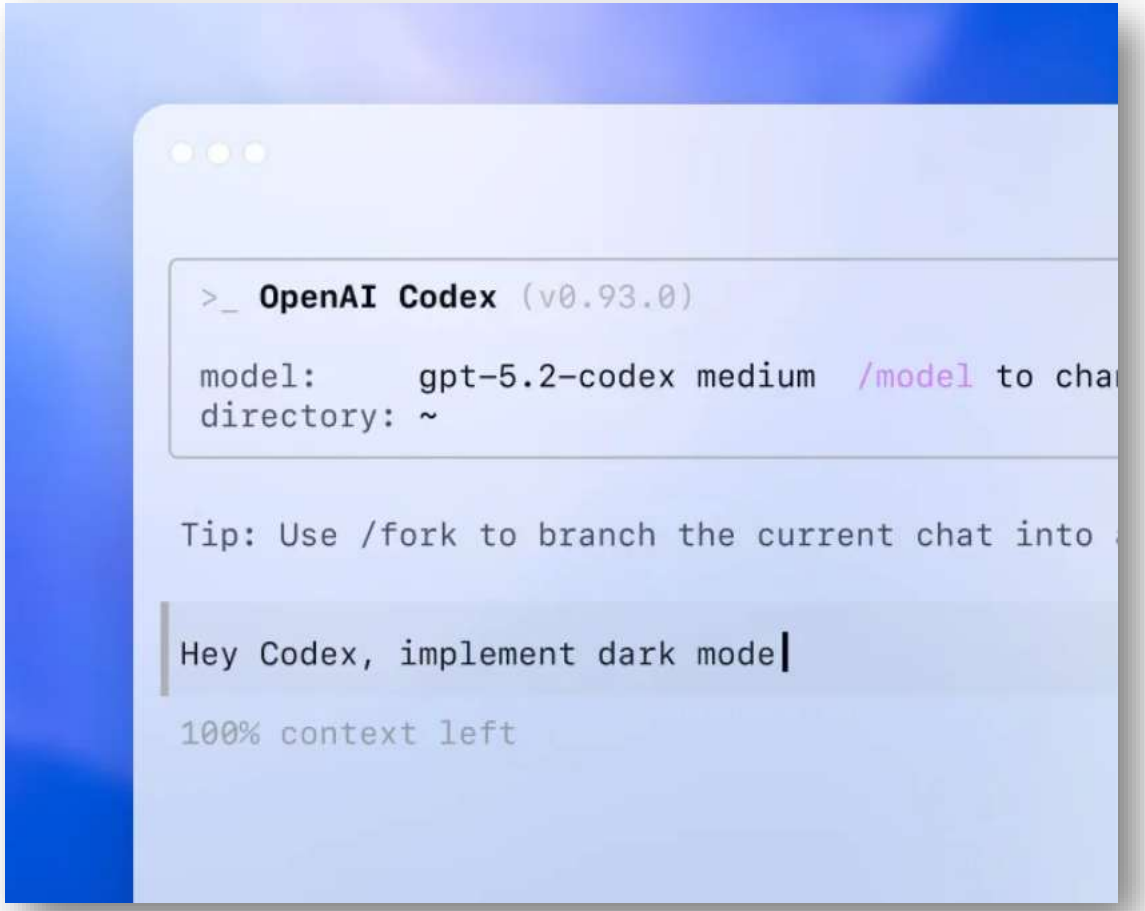
- **深度理解整个代码库，**自主多文件改动、执行命令与 Git 操作。
- **批量扩展性极强：**斜杠命令、记忆、子代理、计划模式、Hooks、MCP、自定义技能。
- **擅长：**大型代码库多步重构、自动化开发流程、复杂调试、可深度定制的工程化使用。



其还具有 **IDE 插件** 与 **桌面工作台** 形态，真正做到了同一引擎，多种形态。

Codex CLI

闭源 OpenAI 模型



- **功能灵活，**可本地沙箱执行代码；提供不同审批 / 自治档位（从每步确认到较高自治）。
- 与 OpenAI 模型 / 生态及 ChatGPT 中的 Codex **云端任务联动**，是 Codex 家族的命令行成员。
- **擅长：**偏好 OpenAI 模型的开发者、需要开源可审计工具的团队、终端内的自动化编码。

概念点：**审批档位**，意味着自治程度可调，是平衡效率与安全的重要旋钮。

Gemini CLI

开源 慷慨免费额度 Gemini

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- **超大上下文窗口**（百万级 token），利于整库理解；个人使用有慷慨免费额度。
- 内置 Google 搜索接地、支持 MCP；ReAct 推理-行动循环；**与 Google 生态结合**。
- **擅长：**超大代码库/长文档整体分析、对成本敏感的学习与个人项目、需联网检索信息的开发。



两大特点 **上下文大 + 免费友好**，比较适合学生群体课后上手。

OpenCode

开源 模型无关



- **模型 / 厂商无关：**可自由接入 Anthropic / OpenAI / Google / 本地模型等任意提供方。
- 漂亮的 TUI、客户端-服务端架构、LSP 集成；**完全开源**、可自托管、高度可定制。
- **擅长：**不想被单一厂商锁定、想混用多家模型、追求完全可控 / 自托管的开发者与团队。

属于第一类中最自由的代理式编程工具，其核心特点在于模型选择自由，以及支持开源与自托管部署。

四款命令行 Agent 对比

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

维度	Claude Code CLI	Codex CLI	Gemini CLI	OpenCode
背后模型	Claude	OpenAI	Gemini	**任意（自选）**
开源	否	是	是	是
上下文规模	大	中-大	**超大（百万级）**	取决于所选模型
生态/扩展	**很强（MCP/技能/子代理）**	OpenAI生态	Google生态/搜索	开源社区/可自托管
免费友好	订阅制	按量/订阅	**慷慨免费额度**	自带Key
适合人群	工程化&可定制	OpenAI生态用户	大上下文&省钱	不锁厂商

深度工程化 & 可定制 → Claude Code CLI

OpenAI 生态 → Codex CLI

大上下文 & 节约资金 → Gemini CLI

不锁厂商 & 自托管 → OpenCode

编辑器原生 Agent



Cursor



Trae



Antigravity

共同点：均把 Agent 作为产品的核心卖点，深度融入编辑、调试、可视化的开发环境。

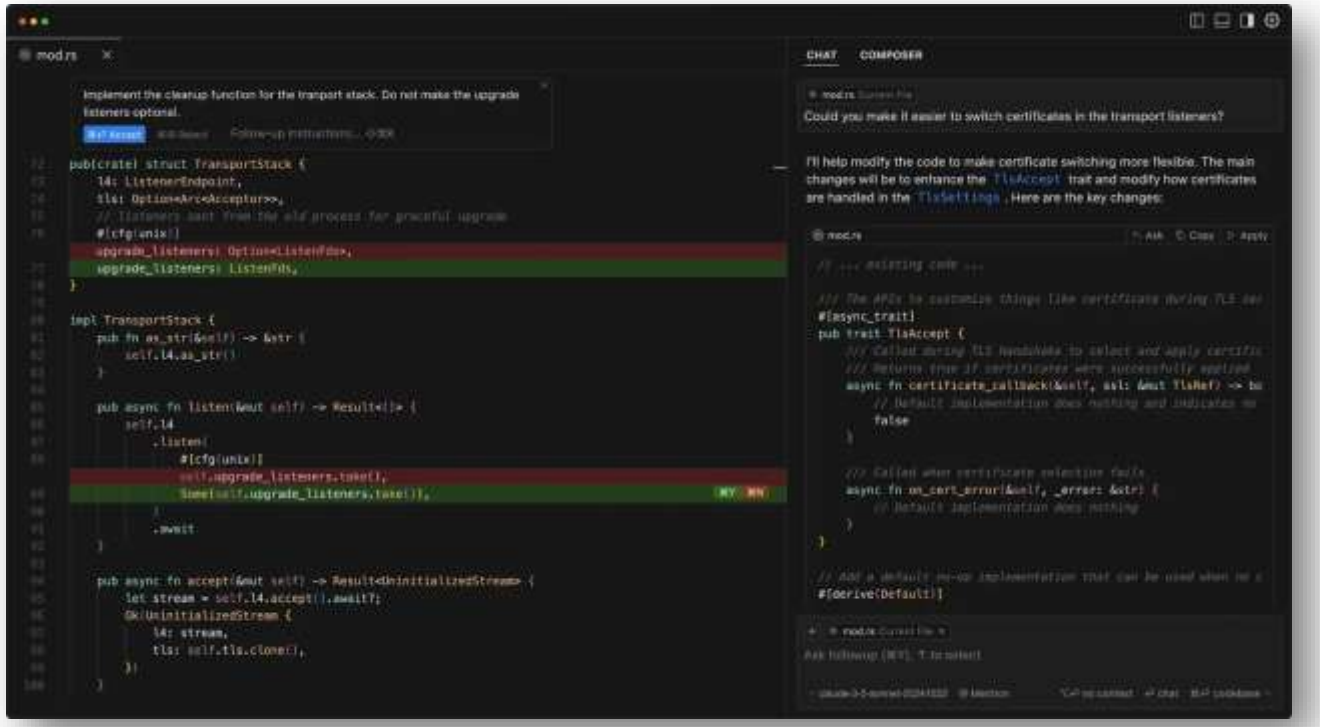
关注差异：成熟度、免费策略、是否 Agent 优先。

Cursor

闭源　多模型可选

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

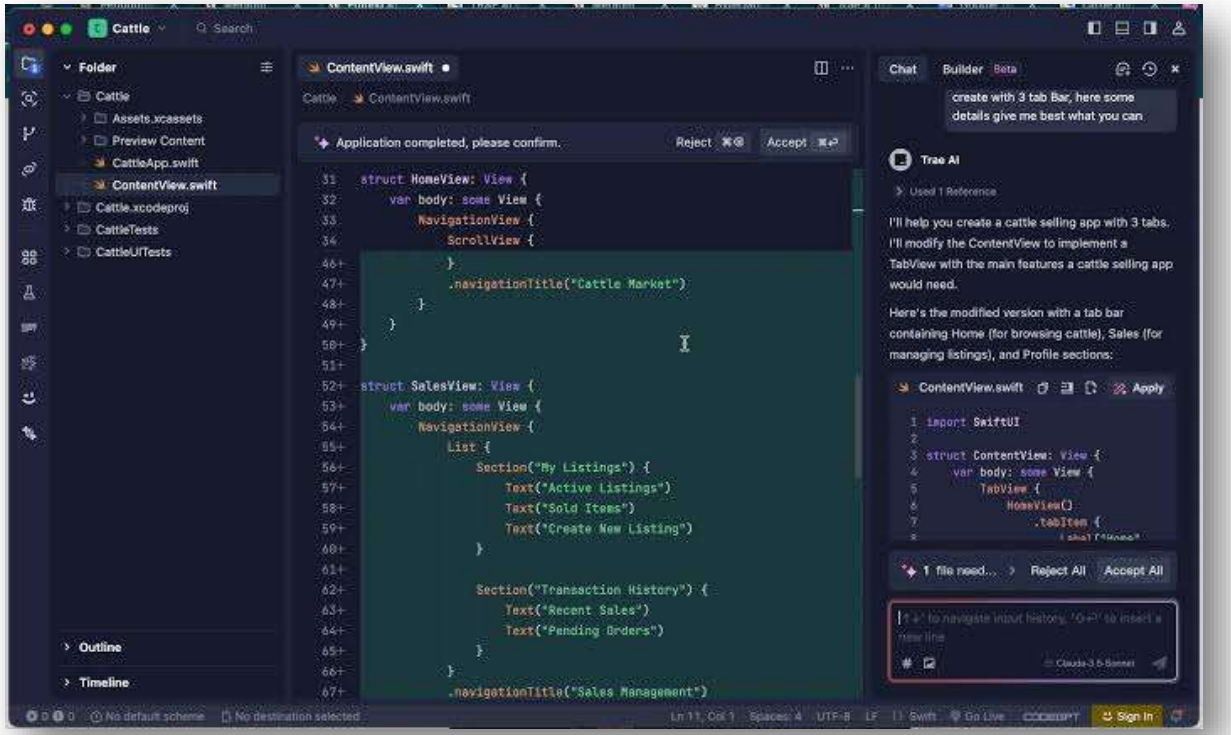
- Tab 补全、Cmd-K 行内编辑、对话面板、**Composer / Agent 模式**（自主多文件改动 + 可视化 diff）、后台代理。
- **可选多模型**（Claude / GPT / Gemini 及自研 Composer 模型）；体验成熟、响应快。
- **擅长：**日常全栈 / 前端开发、需频繁人工审阅微调的工作、从 VS Code 平滑迁移的开发者。



既能像传统编辑器精修，也可以一键交给 Composer 自动改一系列文件，人机协作紧密。

Trae

高性价比　新手友好



- 基于 VS Code 的 AI 原生 IDE，内置 **Builder / SOLO 等 Agent 模式**，可端到端完成需求。
- 曾以可免费 / 低成本使用较强模型吸引用户；**界面友好、中文支持好**、门槛低
- **擅长：**学生与初学者入门、个人项目快速搭建、对成本敏感的尝鲜者。

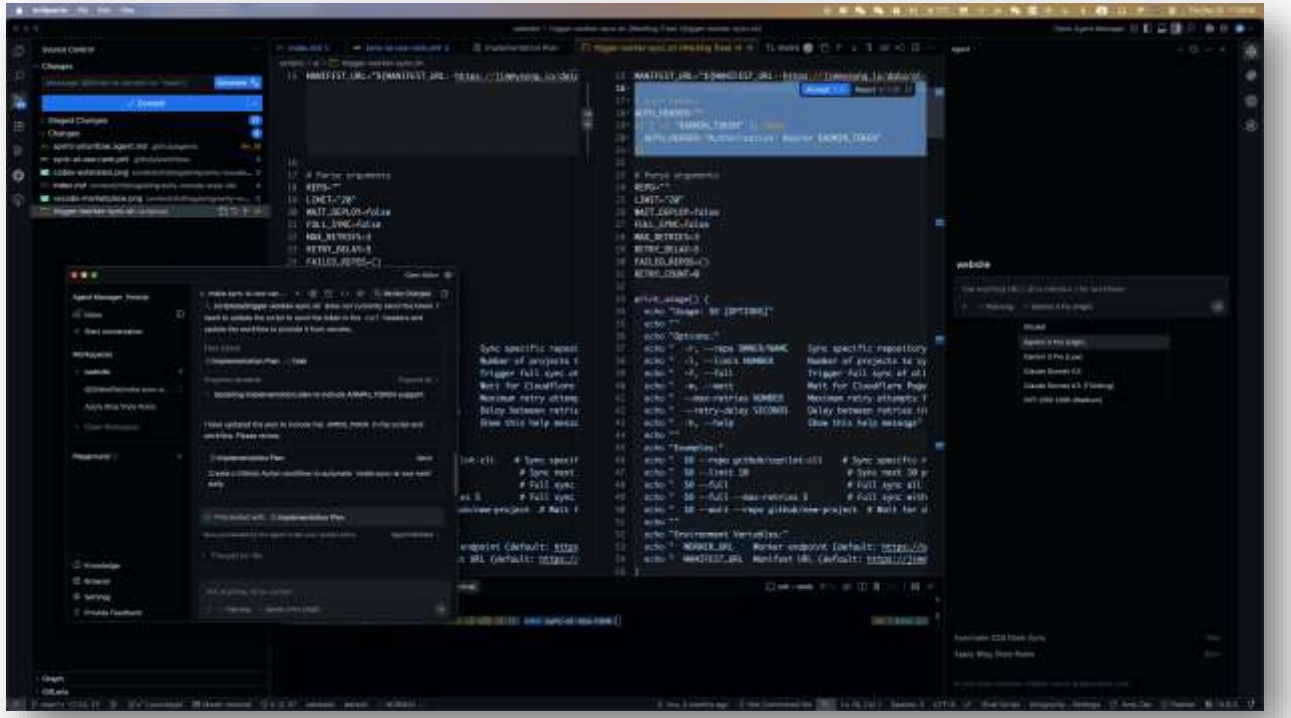
深度融合了大语言模型，支持代码自动补全、自然语言生成代码、智能调试与多智能体。

Antigravity

Agent 优先　Gemini 3

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- **代理优先开发平台**：Agent 管理器可异步调度多个 agent，开发者像指挥者。
- 强调可验证**工件（Artifacts）**（任务清单、截图、浏览器录屏）；可驱动浏览器执行验证；多模型可选。
- **擅长：**以 Agent 为中心的开发范式、需可追溯验证的自动化任务、前端 + 浏览器联动的端到端开发。



将第二类代理式编程向第四类代理式编程（多 Agent 编排）方向延伸，**融合分类边界。**

三款编辑器原生 Agent 对比

维度	Cursor	Trae	Antigravity
成熟/流行度	**高（主流标杆）**	中（增长快）	新（较晚推出）
免费友好	订阅为主	**高（性价比）**	预览期免费
Agent自治	中-高（Composer）	中-高（SOLO）	**高（Agent优先）**
特色	成熟体验/自研模型	易用/低门槛	工作可验证/浏览器控制
适合人群	主流首选	新手低成本	拥抱Agent新范式

趋势：编辑器原生 Agent 正从AI 增强的编辑器
走向Agent 优先的平台（Cursor → Antigravity 的演进方向）。

IDE 插件 Agent

Claude Code
(插件形态)

Cline

GitHub Copilot

共同点：以**插件形式**嵌入现有 IDE，不要求更换开发环境。

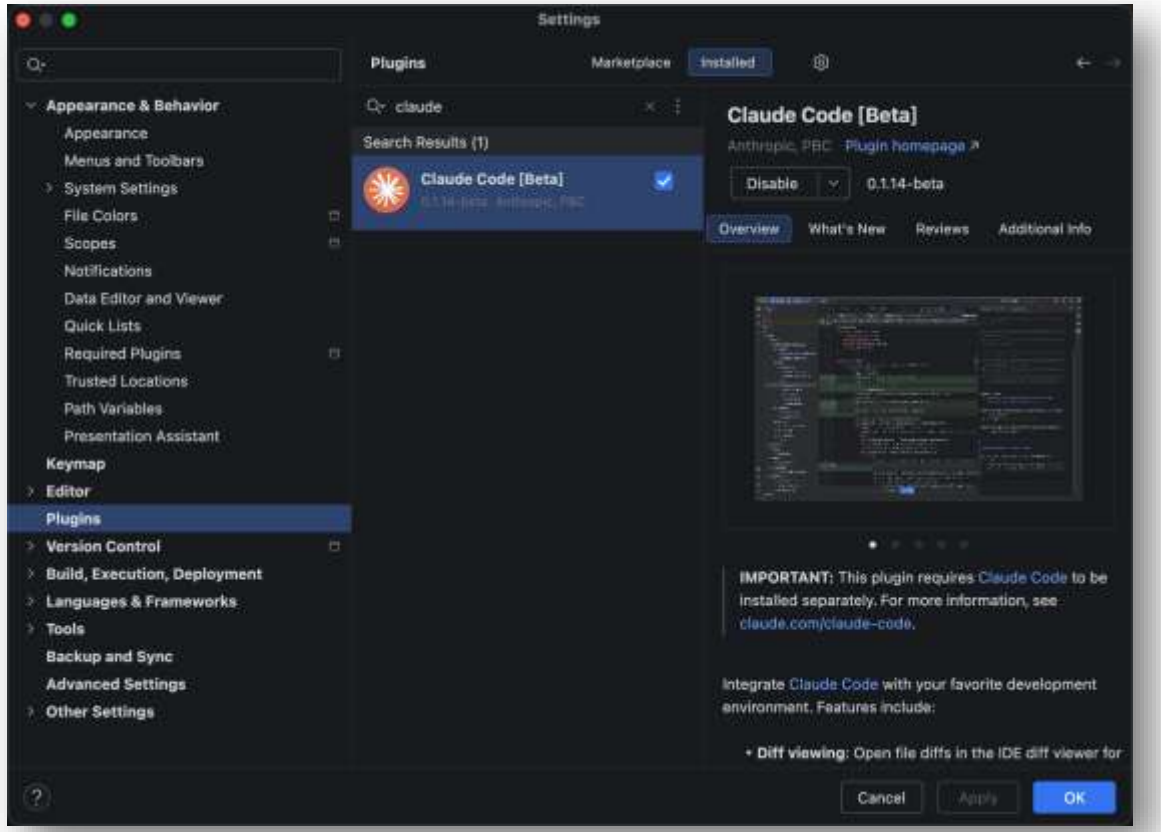
关注差异：这一类产品从传统辅助向Agent演进的过程最为明显，重点留意其 Agent 模式的差异。

Claude Code 插件

闭源 | Claude 模型

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

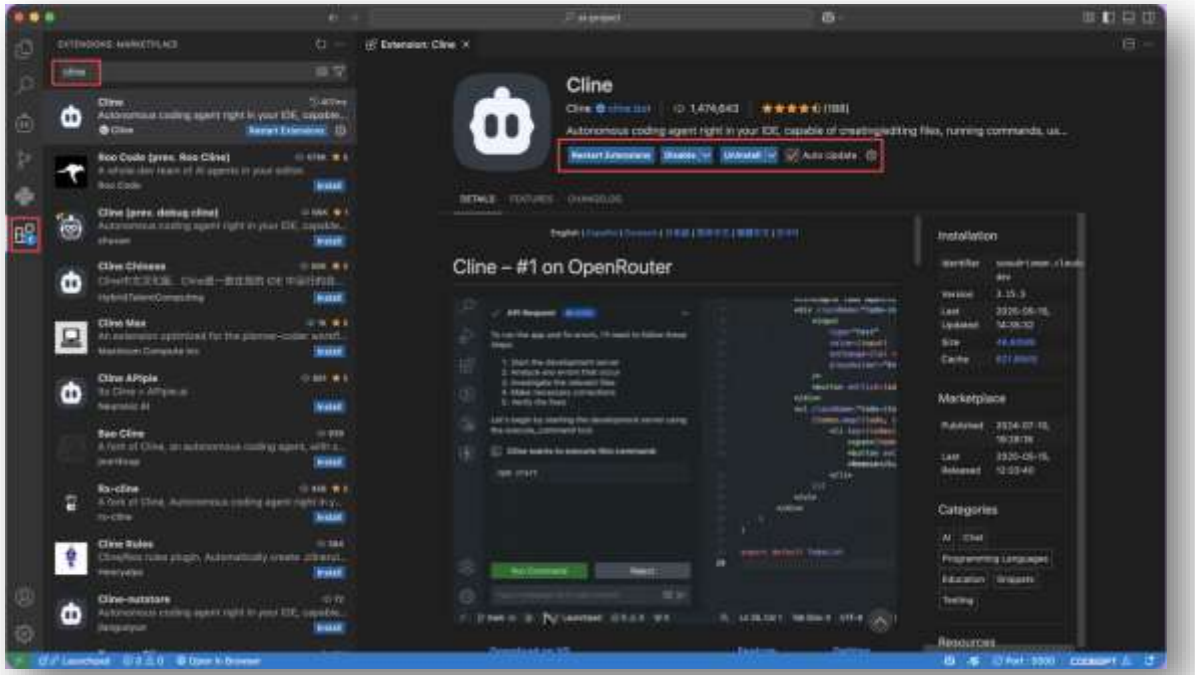
- **将终端 Agent 装进 IDE**：原生 diff 视图、选区上下文、改动可视化审阅。
- 保留 CLI 的**全部能力**（计划模式 / 子代理 / MCP），但交互更图形化。
- **擅长：**习惯 IDE 的人，想要 Claude Code 的强能力与**可视化审阅**。



关键回扣：其与第一类代理式编程的 Claude Code CLI 是**同一工具**的不同外壳。本页只讲插件形态带来的差异，不重复引擎能力。

Cline

开源　模型自由 BYO Key



- 编辑器内自治编码 Agent，**Plan / Act 双模式**（先规划、后执行）。
- **自带模型选择**（接 OpenRouter / 本地等）；支持 MCP、执行命令、浏览器操作；**每步透明可见、需人确认**。
- **擅长：**想要开源可控 + 模型自由、重视每步可审的开发者。

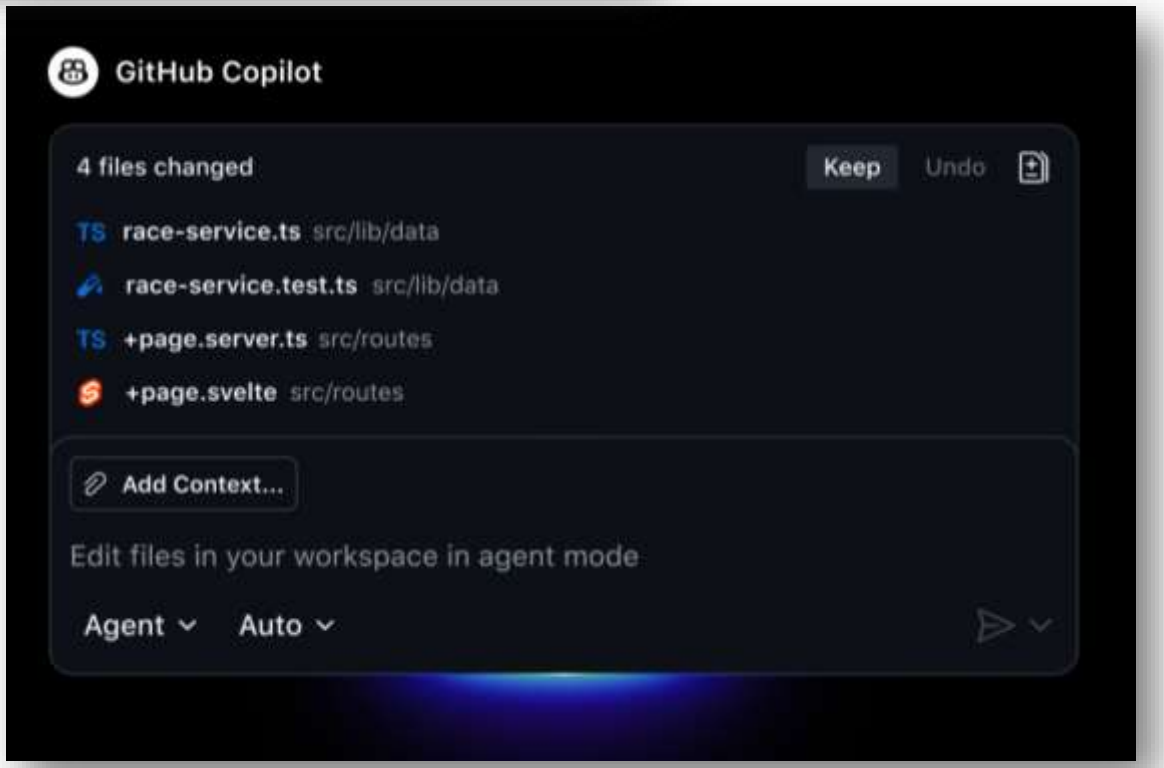
Plan / Act 双模式，是所推崇的**先规划后执行**的产品化体现。

GitHub Copilot

闭源　多平台 · 多模型

演化路径：自动补全 → Copilot Chat 对话 → Copilot Edits 多文件 → Agent 模式 / 编码 Agent

- **生态最广：**深度集成 GitHub 工作流，覆盖 VS Code / JetBrains 等多平台。
- 已从补全演进出 **Agent 模式 / 编码 Agent**；可选多家模型。
- **擅长：**企业团队提效、GitHub 重度用户、在标准 IDE 内渐进引入 AI。



典型的不完全 agentic、但**边界正在模糊，分类是动态**的，工具在向更高自治演进。

三款 IDE 插件对比

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

维度	Claude Code 插件	Cline	GitHub Copilot
开源	否	**是**	否
自治程度	**高**	**高（Plan/Act）**	中（可选Agent模式）
模型自由度	Claude为主	**高（BYO Key）**	多模型可选
生态集成	MCP/技能	MCP/开源生态	**GitHub 深度集成**
企业友好	中-高	中	**高**
适合人群	Claude 能力 + IDE	开源可控 + 模型自由	GitHub 生态 + 企业级

不换环境、平滑升级，是团队规模化落地 AI 编程阻力最小的路径。

桌面端 Agent 工作台

Claude Code Desktop

Codex Desktop

共同点：独立桌面应用，以控制台 / 看板形式**并行管理多个 agent 任务。**

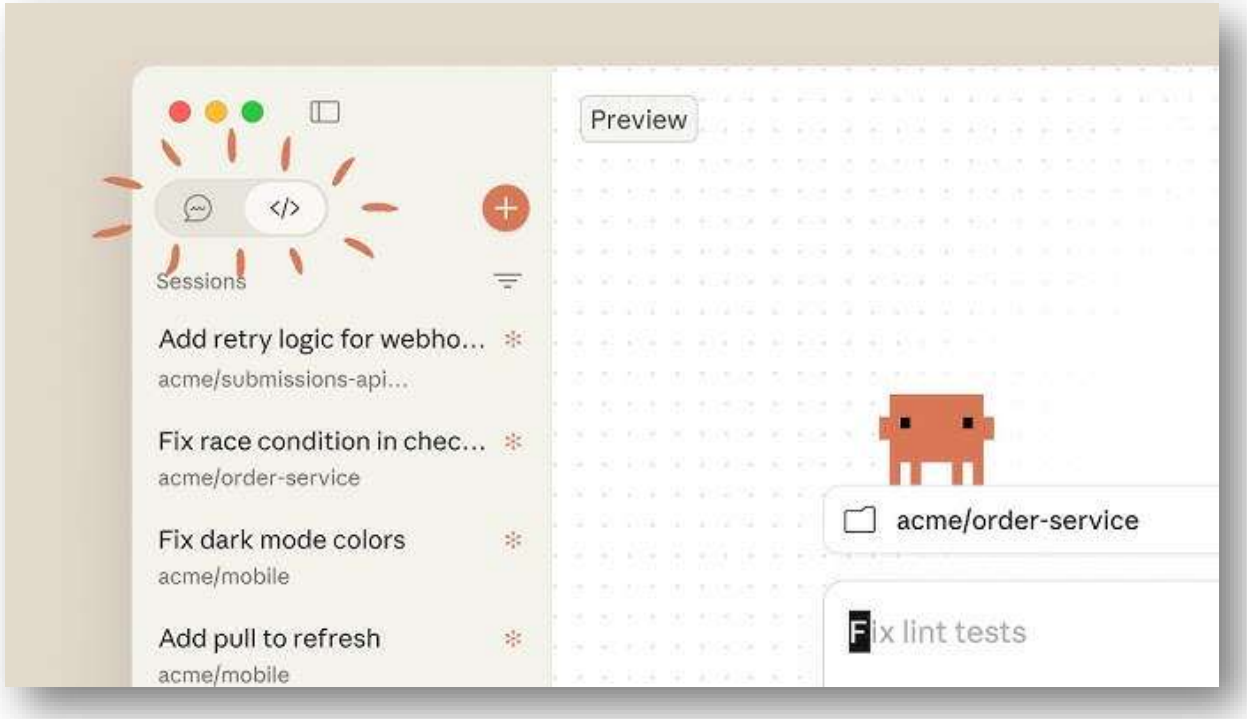
关注差异：当任务数量超出单个 Agent 的处理能力时，就需要通过工作台进行统一编排与协同管理。

Claude Code Desktop

闭源 Claude 模型

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

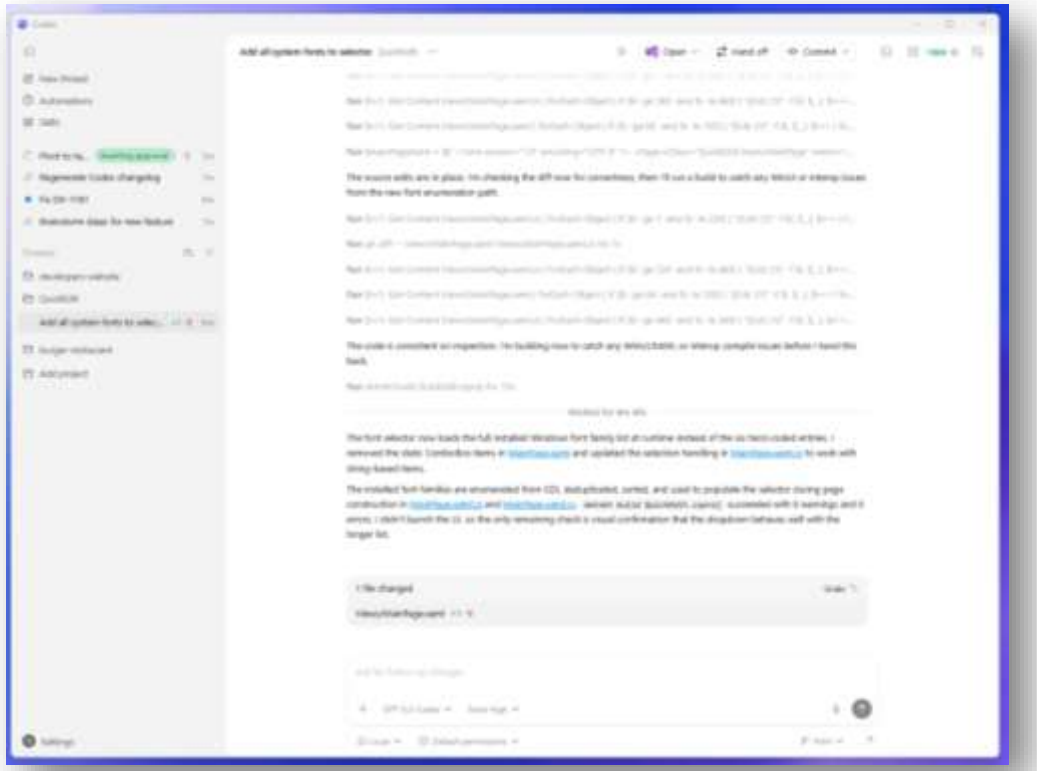
- 图形界面**并行运行多个 agent 会话**（多项目 / 多分支 / 异步作业）、可视化进度与产出。
- 把终端里的强能力变成**看板式的多任务编排**。
- **擅长：**同时推进多个开发任务、长时间异步作业、个人 / 团队的Agent 流水线管理。



Claude Code 的第三种形态：交互单位**从一个对话升级为一组任务。**

Codex Desktop

闭源　OpenAI 模型



- Codex 家族的桌面应用，**管理多个编码任务**；本地与云端执行结合。
- 与 ChatGPT / Codex 生态打通；可视化**跟踪多任务进度与结果**；把云端 Codex Agent 能力带到桌面控制台。
- **擅长：**OpenAI 生态用户的多任务编排、云端长作业管理、桌面端集中调度。

第四类代理式编程工具共性趋势：**异步 + 云端 + 编排**，预示开发协作方式的变化。

各形态 Agent 的底层逻辑

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

命令行

- Claude Code CLI
- Codex CLI
- Gemini CLI
- OpenCode

编辑器原生

- Cursor
- Trae
- Antigravity

IDE 插件

- Claude Code（插件）
- Cline
- GitHub Copilot

桌面工作台

- Claude Code Desktop
- Codex Desktop

逻辑一： 同一引擎可跨形态，Claude Code 横跨 命令行 / IDE 插件 / 桌面工作台，分类按使用入口，不是按厂商。

逻辑二： 选型逻辑。工作形态（终端 / 编辑器 / IDE / 多任务）； 自治度需求；开源需求；是否需要并行编排。按场景选工具。

> 软件开发全流程的**专用 Agent** 与多 **Agent 协同分工**。

除了最主要的编码 Agent 工具，软件开发的每一个环节，几乎都已经有了**专用 Agent**。由人居中调度，**多 Agent 协同**才是未来的趋势。

使用一组各具专长的 Agent 协同参与开发工作



人：拆解任务 · 衔接产物 · 质量把关 · 居中调度（编排者）

把每一步交给最擅长的专用 Agent，再把它们的产物串成一条完整的流水线

需求结构化

Spec Kit · Kiro

原型

v0 · Lovable

流程图 / 设计

Mermaid · Eraser

实现

Claude Code · Cursor

文档

Mintlify · DeepWiki

测试

Qodo · QA Wolf

擅长把需求结构化的 Agent

规格驱动开发（SDD）：把结构化规格作为事实来源，让代码成为规格的产物。



GitHub Spec Kit

GitHub · 开源

Spec→Plan→Tasks→Implement 四阶段；**以宪法文件**承载不变原则；可对接 30+ 编码 Agent。



Kiro

Amazon · 规格驱动 IDE

用 EARS 句式写可测验收标准；agent hooks 自动化；写码前用形式化方法找出歧义与矛盾。



ChatPRD

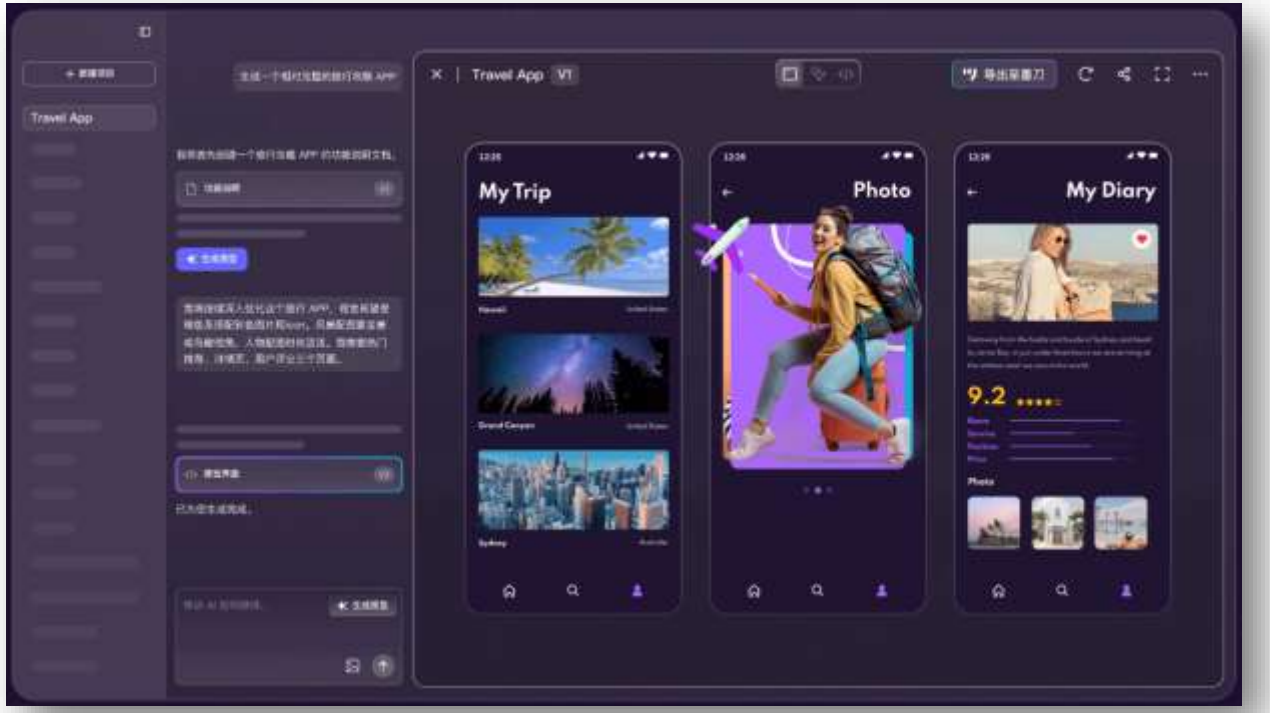
面向产品经理

AI 撰写 / 完善产品需求文档（PRD），把模糊想法整理成清晰、可执行的需求。

需求越结构化、上下文越充分，后续所有 Agent 的产出越可靠。

擅长快速产出原型的 Agent

Lovable



一句话即可生成可运行的界面或应用，**先做出可点击的原型再迭代**，是敏捷 MVP的理念。

- **v0**：擅长生成**规范的 React/Next.js 前端**，便于交付工程团队。
- **Bolt.new**：浏览器内 WebContainers，**速度快**是其主要特点。
- **Lovable**：面向**非技术用户**，自带后端与鉴权的全栈生成。
- **Replit Agent**：自带数据库与部署，适合**带数据**的内部工具。

快速产出原型：用于需求验证、教学演示、Demo 与 MVP 的快速搭建。

擅长画图的 Agent



Mermaid

开源 · 代码即图

文本描述流程图 、 时序图 、 ER 图，GitHub、Markdown 直接渲染，可在 PR 中评审。



Eraser · DiagramGPT

技术图 + 文档

由自然语言或代码生成技术图，集成 GitHub / Notion / VS Code。



Excalidraw + MCP

手绘风白板

配合 MCP 让编码 Agent 直接读写图文件，使架构图随代码同步演进。



Napkin AI

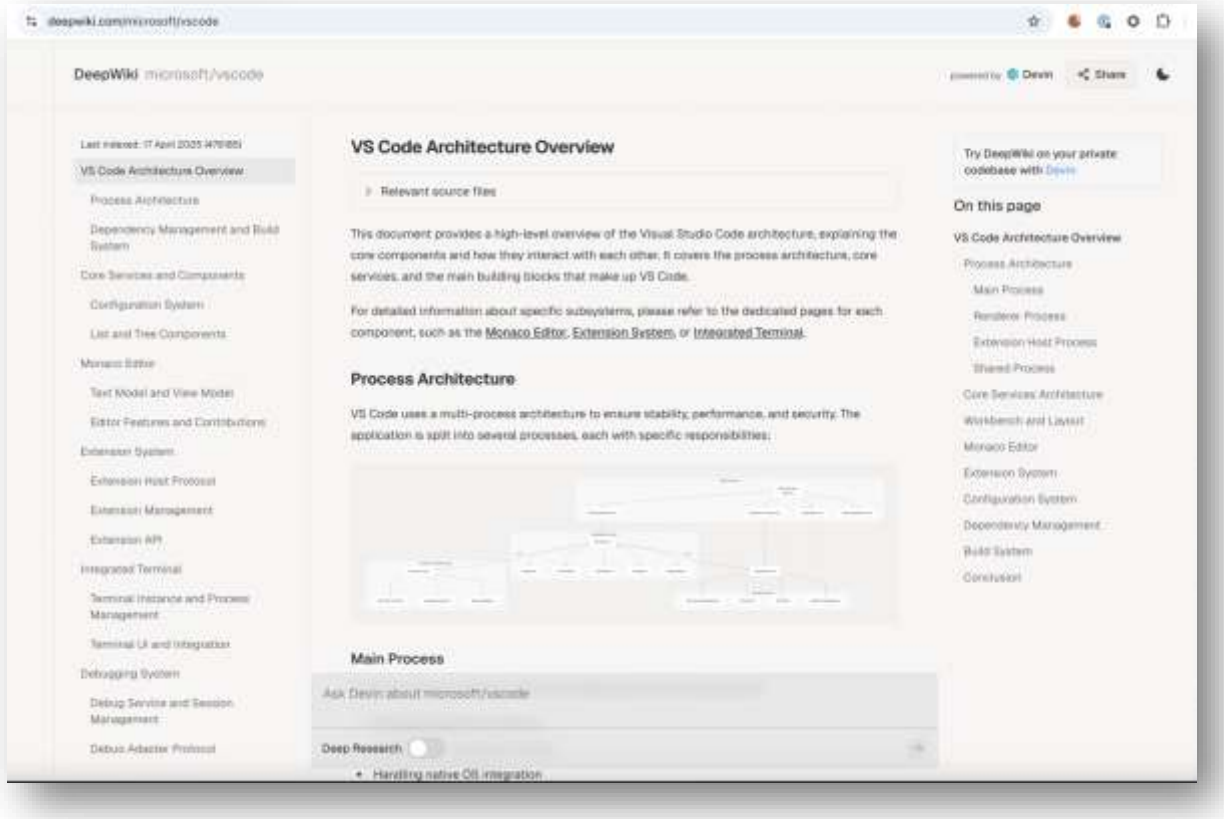
文档配图

文本转可视故事，适合文档 / 博客的精美静态图，偏成品而非可维护结构。

架构图重在长期可维护，优先选可移植、可评审、可随代码更新的方案（如PlantUML）。

擅长撰写文档的 Agent

DeepWiki



文档普遍缺乏持续撰写动力，以及完成后容易过时。文档 Agent 可由**代码变更自动驱动**，实现文档的持续维护。

- **Mintlify**：文档平台，自带可读代码改动、起草更新、**自动提 PR** 的 Agent。
- **DeepWiki**：可以把任意代码仓库自动转成可**对话的 Wiki**，迅速上手项目。
- **Fern**：面向 **API 优先**团队，生成文档与多语言 SDK。

近半文档流量**来自 AI Agent**的程序化访问，文档要同时服务人与Agent两类读者。

擅长做测试的 Agent



Qodo

前身 CodiumAI

结合代码结构（分支、边界）生成单元测试，并强化 PR 的上下文评审。



Diffblue Cover

专注于 Java

自动生成 JUnit 单元测试、追求高覆盖率；在 Java 单测层表现突出。



QA Wolf

QA 即服务

托管式大规模生成 / 维护 Playwright 端到端测试，并对覆盖率作出承诺。



Meticulous

会话录制回放

录制真实用户会话并回放以发现回归，实现无需手写测试，推进自动化程度。

测试是 **Agent 自治**的护栏；但仍需人工评审覆盖设计、安全与合规。

全流程各步骤专用 Agent 对比

开发环节	代表性专用 Agent
需求结构化	GitHub Spec Kit · Kiro · ChatPRD
原型	v0 · Bolt.new · Lovable · Replit Agent
流程图/设计	Mermaid · Eraser · Excalidraw + MCP
实现	**Claude Code • Cursor • Copilot …（通用编码 Agent）**
文档	Mintlify · DeepWiki · Fern
测试	Qodo · Diffblue · QA Wolf · Meticulous
评审 / 安全	Qodo · CodeRabbit · Greptile …

根据软件开发的每一个步骤的特点，选择对应的 Agent ，加快效率。



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

4 代理式编程的开发流程

从代码编写者到 Agent 指挥者

在当今时代，开发者的新身份正在转变为架构师、审稿人与指挥官。工作重心也转向三项更关键的能力：

- **把需求表达清楚**——准确描述意图，并合理拆解任务。
- **把方案规划明确**——确定架构、边界与技术选型。
- **把产出审核到位**——完成审阅、纠偏，并对最终结果负责。



一个常见误区是：AI 的到来**并不意味着不再需要理解技术**。恰恰相反，**判断力、架构能力与工程素养**将变得更加重要。

代理式开发的一般流程



在上述循环中两个关键点：

- 上下文工程，让 Agent 理解项目；
- 人必须全程审阅，**关键决策由人把关**。

敏捷开发与代理式编程

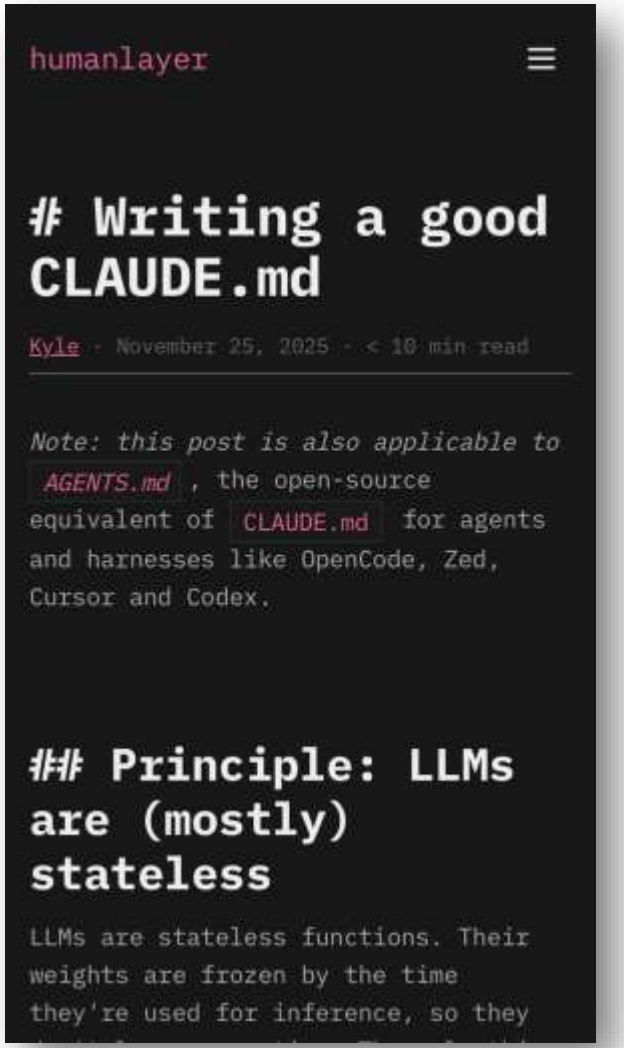
敏捷开发实践		代理式编程中的对应
用户故事 / 需求	↔	给 Agent 的需求描述 / 规格（含验收标准）
Sprint 迭代	↔	Agent 的"规划 - 执行 - 评审"循环
持续集成 / 测试	↔	Agent 自动跑测试、按结果自修
增量交付 / MVP	↔	快速出可运行原型，再迭代
看板 / 每日站会	↔	工作台类的多任务编排与进度可视化
回顾会议	↔	复盘 Agent 产出、优化上下文与提示

敏捷的一个重要特点是小步、增量、快速反馈、拥抱变化；与 Agent 闭环高度同构；它**把迭代周期从周压缩到小时甚至分钟**。

步骤一：需求规格，将需求表达清楚

- **上下文工程：**用项目说明文件（如 CLAUDE.md / 规则文件）固化技术栈、规范、目录结构与约束。
- **规格驱动：**把模糊想法写成清晰的需求 + 验收标准（= 用户故事 + Definition of Done）。
- **提供背景：**接口文档、示例、参考实现一并给到。

Garbage in, garbage out，提示词表述模糊，Agent 会难以理解你的需求。上下文质量，决定产出质量。清晰提出需求是新的核心技能。



步骤二：规划与设计，Agent 给出方案，手动审阅



A

计划模式（Plan Mode）

动代码前，先让 Agent 给出打算怎么做，人确认后再执行（对应 Cline 的 Plan/Act、Claude Code 计划模式）。

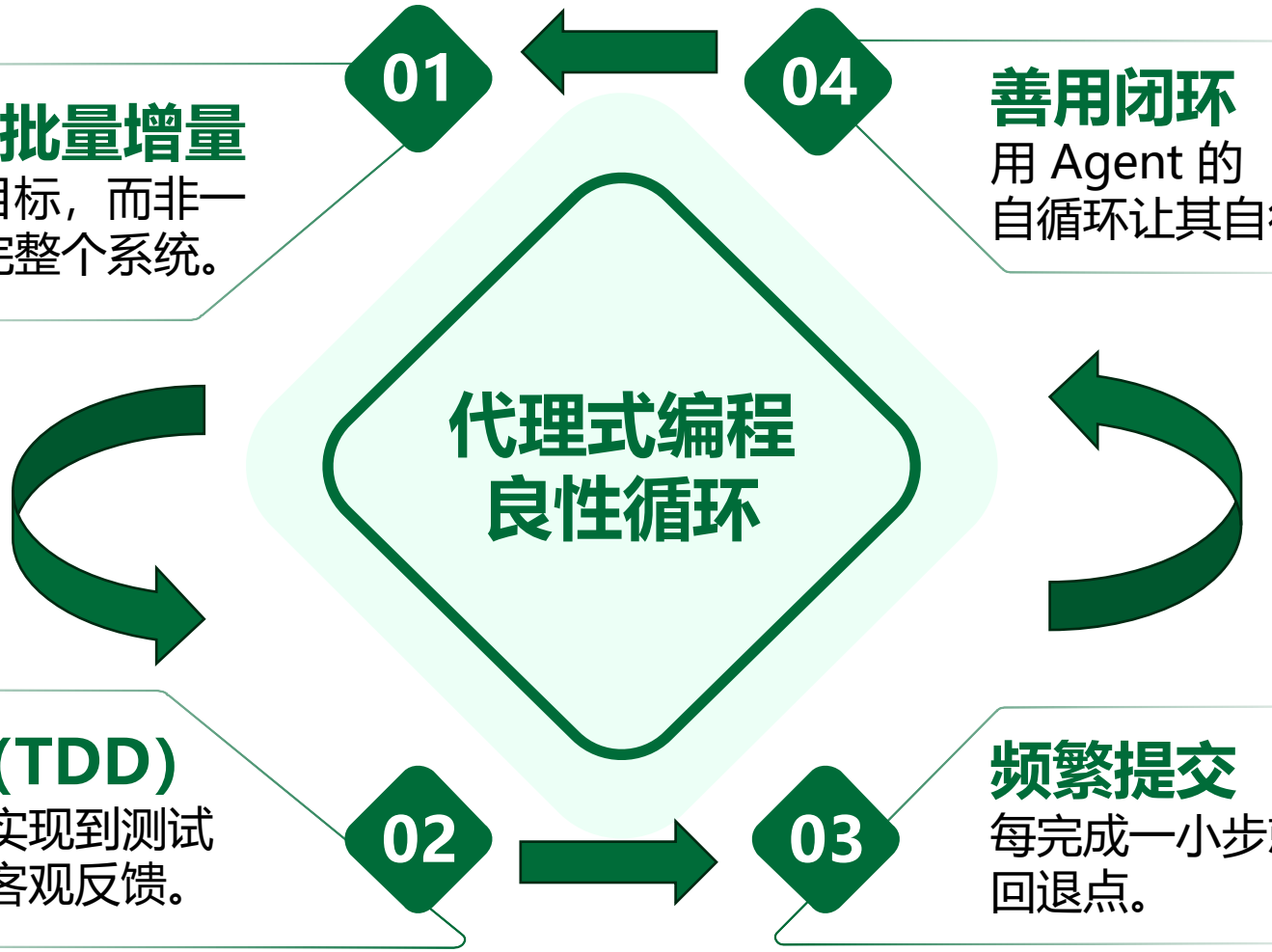
B

架构拆解

把大任务拆成可独立验证的小步，逐个击破。

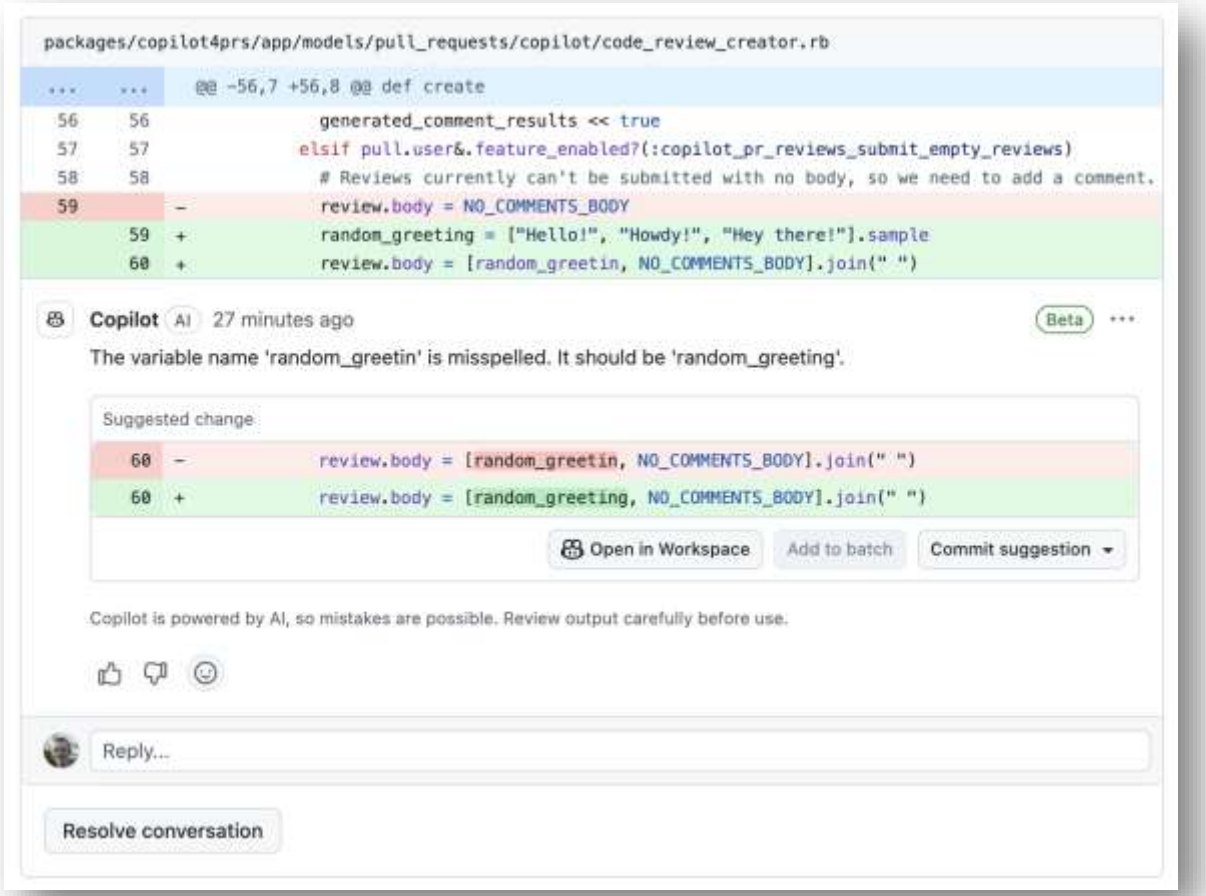
先规划后执行：方案阶段纠错成本最低，越往后改成本越高，这是软件工程的经典认知。

步骤三：编码实现，一次一小步，每步可验证



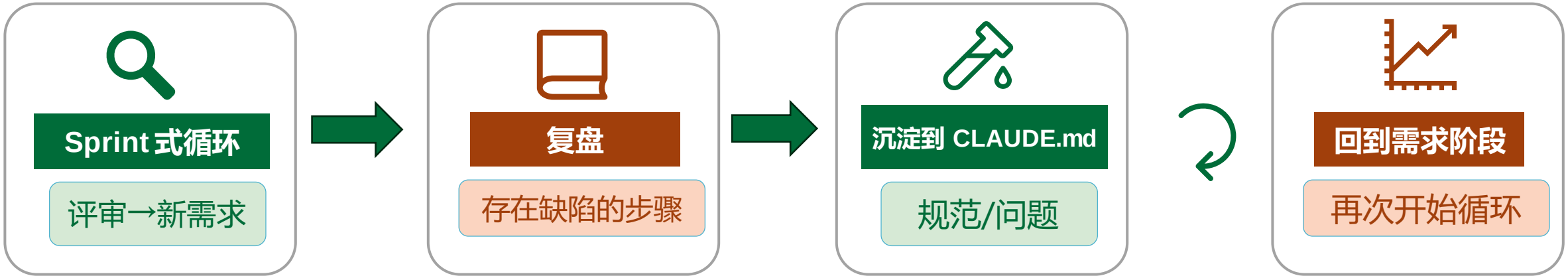
步骤四：测试与评审，Agent 编码后，手动评审

- **自动化测试护栏：**单元/集成测试 + CI，让对错有客观反馈，Agent 可据此自修。
- **代码评审：**人必须审逻辑、安全、可维护性、是否符合规格，对结果负责的是人。
- **善用可验证工件：**测试报告、运行截图、Artifacts 提升可信度。
- **警惕幻觉：**提防表面上可以运行、实际却存在隐患的结果。



Agent 负责产出，人负责把关；速度来自 Agent，质量来自人。

步骤五：迭代与交付，把经验沉淀回上下文



复盘 问对问题

- 需求不明确 / 上下文缺失；寻找根因。

沉淀 让Agent更了解项目

- 把遇到的问题、定下的规范写回上下文文件，下一轮避免再次出现，这是代理式开发的复利。

端到端示例：做一个个人待办事项 Web 应用

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

1、需求与规格

写需求 + CLAUDE.md（技术栈、验收标准）
如：技术栈 React+FastAPI；实现待办的增删改查，每个接口都要有单元测试

2、规划与设计

Plan 模式出方案、拆任务，人审阅确认后放行

3、编码与实现

TDD 增量实现：先后端 API → 再前端，每完成一步就提交

4、测试与评审

跑测试 + 人工评审 + 修缺陷，确认符合验收标准

5、迭代与交付

加需求迭代（如“加用户登录”），把新规范沉淀回 CLAUDE.md

用好 Agent 的经验

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

要做：

需求写清楚、提供充分上下文

小步提交、保留可回退点

建立测试护栏 + 人工评审

敏感操作审批；不懂代码读懂再合

不做：

一句话甩一个大需求

放任长程自治、全程不审

盲信能跑就行，无测试直接合并

把密钥 / 敏感信息随意交给工具

工具决定能做多快，流程与判断力决定能做多好。

拥抱变化，保持判断

更强自治、多 Agent 协作、与软件工程流程深度融合是大势；

工具会变，而**工程素养与判断力长期有价值。**



Q & A

提问环节

北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

BEIJING INSTITUTE OF TECHNOLOGY

谢谢

授课人：闫波

时　间：2026/9/21